

Gazipur Digital University

Department of Software Engineering

Course Titel : Data Structure and Algorithms

Course Code : CSE-114

Submitted To: Md. Moshir Rahman

Lecturer & Chairman Department of Software
Engineering

Submitted By : Md. Akash Mia

ID:2303039

Session:2023-2024

Date of Submission:August 29,2025

1. Insertion Sort :

◆ Explanation:

Insertion Sort is a simple comparison-based sorting algorithm. It works by dividing the array into a **sorted** and **unsorted** part. One by one, elements from the unsorted part are picked and inserted into the correct position in the sorted part.

Example:

Input: [5, 3, 4, 1]

Step 1: [3, 5, 4, 1]

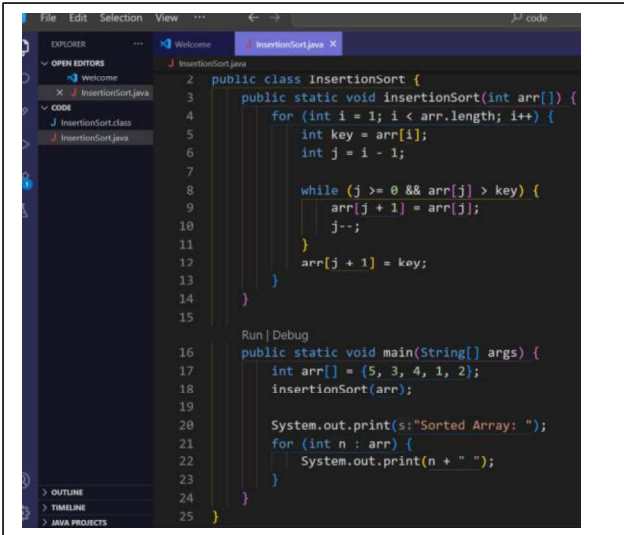
Step 2: [3, 4, 5, 1]

Step 3: [1, 3, 4, 5]

◆ Time Complexity:

- Best Case: **$O(n)$** (if already sorted)
- Worst Case: **$O(n^2)$**
- Average Case: **$O(n^2)$**
- Space Complexity: **$O(1)$**

◆ Java Code:n Input



```
1 public class InsertionSort {
2     public static void insertionSort(int arr[]) {
3         for (int i = 1; i < arr.length; i++) {
4             int key = arr[i];
5             int j = i - 1;
6
7             while (j >= 0 && arr[j] > key) {
8                 arr[j + 1] = arr[j];
9                 j--;
10            }
11            arr[j + 1] = key;
12        }
13    }
14
15    Run | Debug
16    public static void main(String[] args) {
17        int arr[] = {5, 3, 4, 1, 2};
18        insertionSort(arr);
19
20        System.out.print(s:"Sorted Array: ");
21        for (int n : arr) {
22            System.out.print(n + " ");
23        }
24    }
25 }
```

Output



```
[Running] cd "d:\DSA\code\" && javac InsertionSort.java && java InsertionSort
Sorted Array: 1 2 3 4 5
[Done] exited with code=0 in 2.926 seconds
```

◆ 2. Merge Sort :

◆ Explanation:

Merge Sort is a Divide and Conquer algorithm.

- The array is divided into two halves.
- Each half is recursively sorted.
- Finally, the two halves are merged into a sorted array.

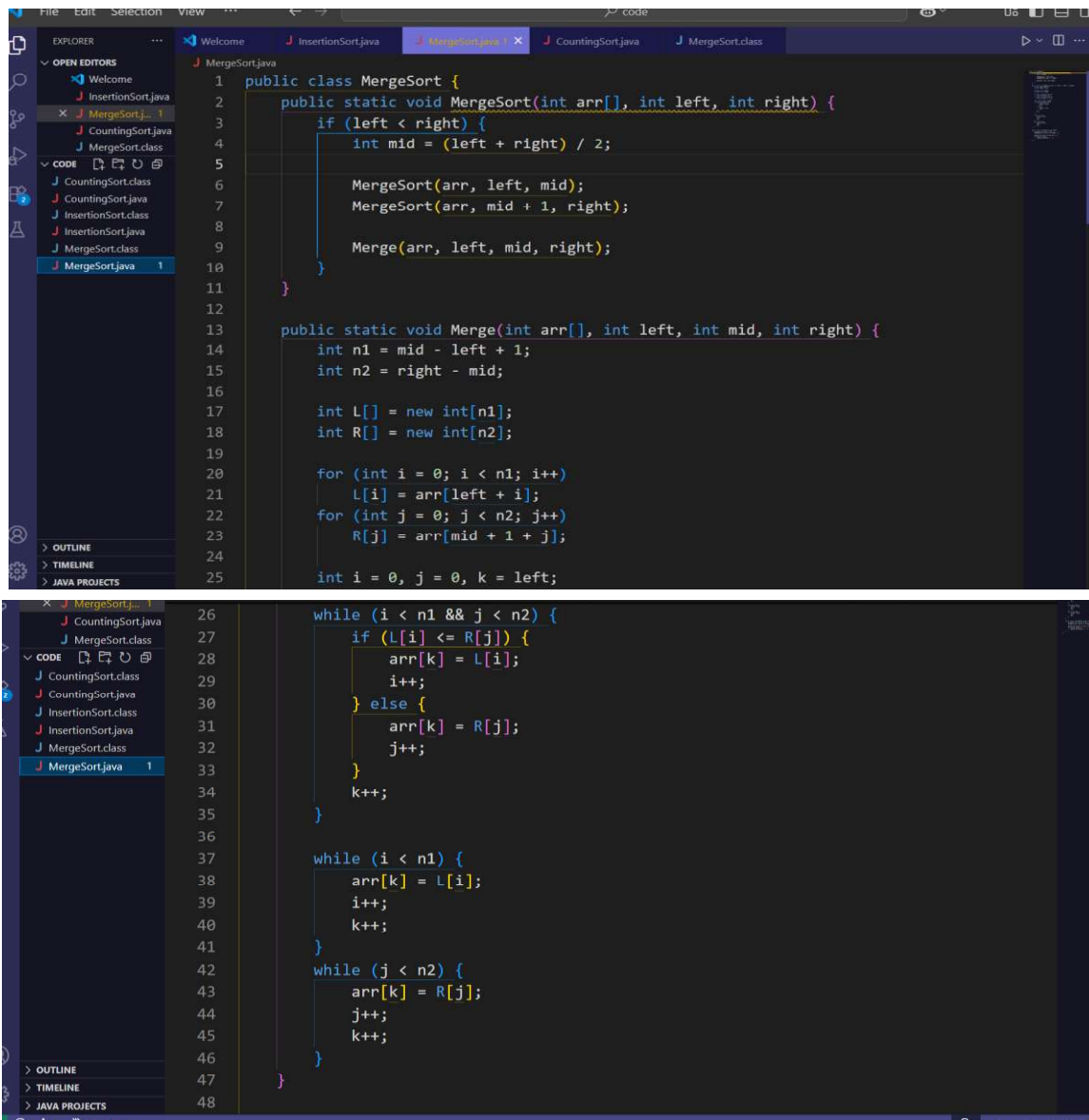
Example:

[5, 2, 4, 7, 1] → split → [5, 2], [4, 7, 1] → merge → [1, 2, 4, 5, 7]

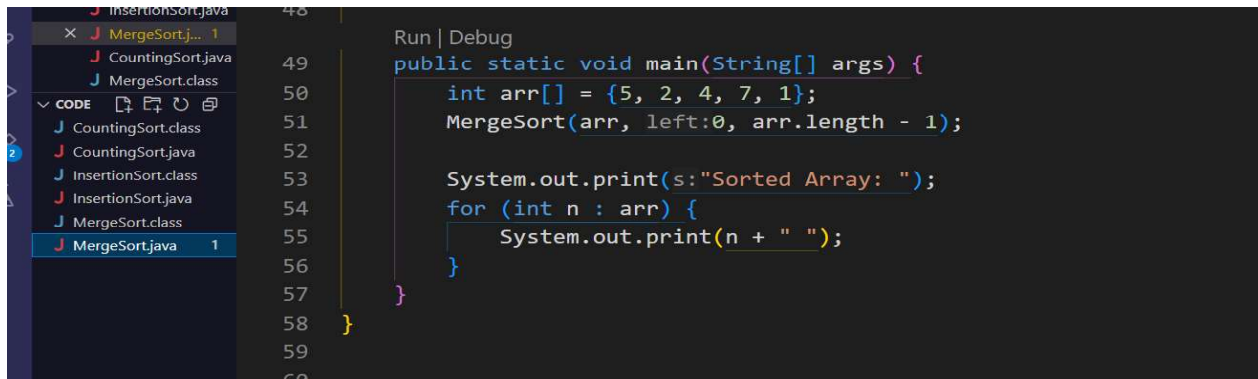
◆ Time Complexity:

- Best Case: $O(n \log n)$
- Worst Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Space Complexity: $O(n)$

◆ Java Code:



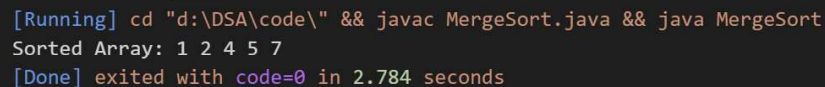
```
1 public class MergeSort {
2     public static void MergeSort(int arr[], int left, int right) {
3         if (left < right) {
4             int mid = (left + right) / 2;
5
6             MergeSort(arr, left, mid);
7             MergeSort(arr, mid + 1, right);
8
9             Merge(arr, left, mid, right);
10        }
11    }
12
13    public static void Merge(int arr[], int left, int mid, int right) {
14        int n1 = mid - left + 1;
15        int n2 = right - mid;
16
17        int L[] = new int[n1];
18        int R[] = new int[n2];
19
20        for (int i = 0; i < n1; i++)
21            L[i] = arr[left + i];
22        for (int j = 0; j < n2; j++)
23            R[j] = arr[mid + 1 + j];
24
25        int i = 0, j = 0, k = left;
26
27        while (i < n1 && j < n2) {
28            if (L[i] <= R[j]) {
29                arr[k] = L[i];
30                i++;
31            } else {
32                arr[k] = R[j];
33                j++;
34            }
35            k++;
36        }
37
38        while (i < n1) {
39            arr[k] = L[i];
40            i++;
41            k++;
42        }
43
44        while (j < n2) {
45            arr[k] = R[j];
46            j++;
47            k++;
48        }
49    }
50 }
```

A screenshot of an IDE. On the left, a file explorer shows a project with files: InsertionSort.java, MergeSort.java (selected), CountingSort.java, MergeSort.class, CountingSort.class, InsertionSort.class, and MergeSort.class. The main editor displays the code for MergeSort.java. The code includes a main method that initializes an array {5, 2, 4, 7, 1}, calls MergeSort(arr, 0, arr.length - 1), and prints the sorted array. The code is as follows:

```
public static void main(String[] args) {
    int arr[] = {5, 2, 4, 7, 1};
    MergeSort(arr, left:0, arr.length - 1);

    System.out.print(s:"Sorted Array: ");
    for (int n : arr) {
        System.out.print(n + " ");
    }
}
```

Output:

A terminal window showing the execution of the MergeSort.java program. The output is as follows:

```
[Running] cd "d:\DSA\code\" && javac MergeSort.java && java MergeSort
Sorted Array: 1 2 4 5 7
[Done] exited with code=0 in 2.784 seconds
```

3. Counting Sort:

◆ **Explanation:** Counting Sort is a non-comparison sorting algorithm.

It works by counting the occurrences (frequency) of each element and then arranging elements based on that frequency.

Example:

Input: [4, 2, 2, 8, 3]

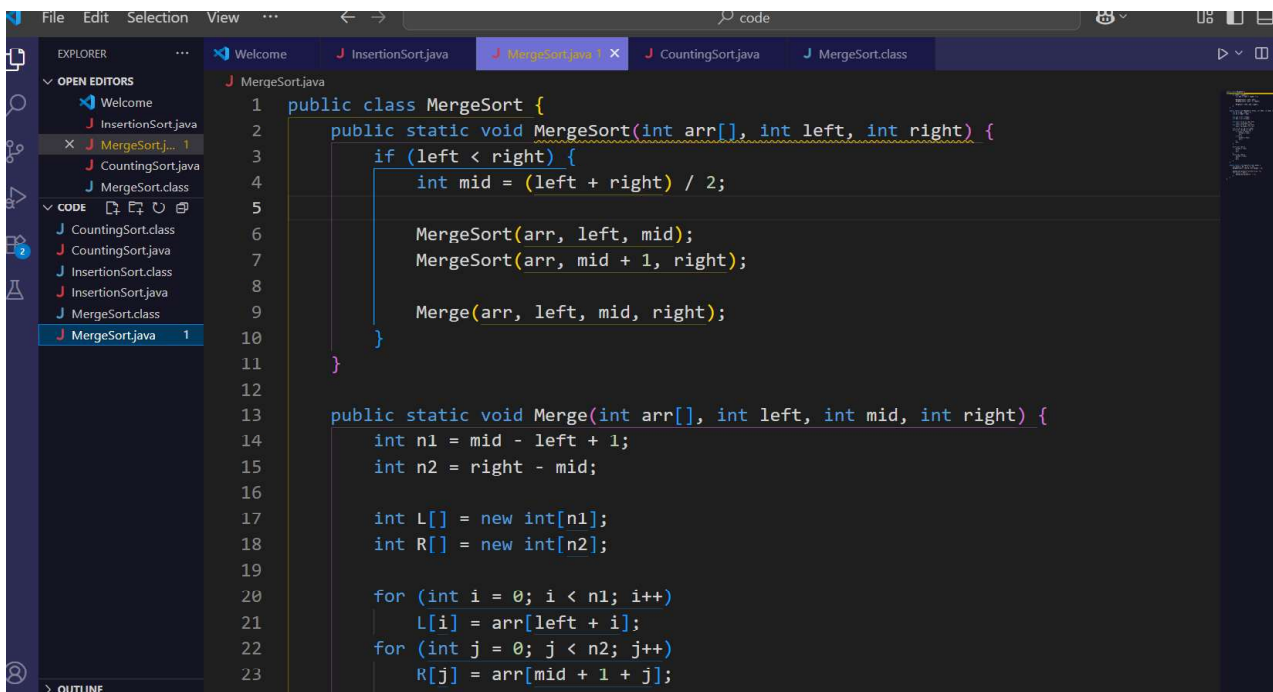
Count Array: [0, 0, 2, 1, 1, 0, 0, 0, 1]

Sorted Output: [2, 2, 3, 4, 8]

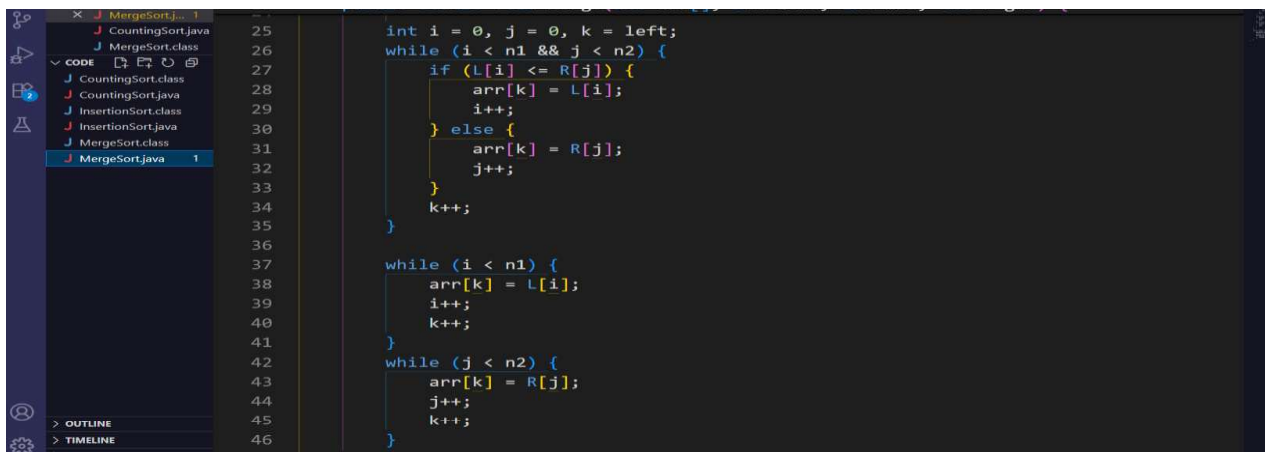
◆ Time Complexity:

- Best Case: $O(n + k)$
- Worst Case: $O(n + k)$
- Average Case: $O(n + k)$
- Space Complexity: $O(k)$ (k = maximum value in the array)

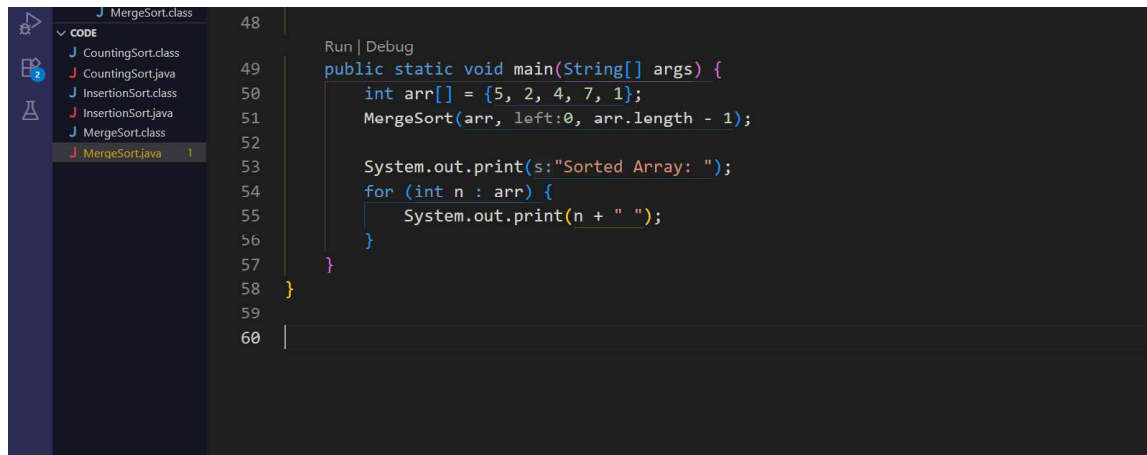
◆ Java Code:



```
1 public class MergeSort {
2     public static void MergeSort(int arr[], int left, int right) {
3         if (left < right) {
4             int mid = (left + right) / 2;
5
6             MergeSort(arr, left, mid);
7             MergeSort(arr, mid + 1, right);
8
9             Merge(arr, left, mid, right);
10        }
11    }
12
13    public static void Merge(int arr[], int left, int mid, int right) {
14        int n1 = mid - left + 1;
15        int n2 = right - mid;
16
17        int L[] = new int[n1];
18        int R[] = new int[n2];
19
20        for (int i = 0; i < n1; i++)
21            L[i] = arr[left + i];
22        for (int j = 0; j < n2; j++)
23            R[j] = arr[mid + 1 + j];
```



```
25        int i = 0, j = 0, k = left;
26        while (i < n1 && j < n2) {
27            if (L[i] <= R[j]) {
28                arr[k] = L[i];
29                i++;
30            } else {
31                arr[k] = R[j];
32                j++;
33            }
34            k++;
35        }
36
37        while (i < n1) {
38            arr[k] = L[i];
39            i++;
40            k++;
41        }
42        while (j < n2) {
43            arr[k] = R[j];
44            j++;
45            k++;
46        }
47    }
```



```
48  
49  
50  
51  
52  
53  
54  
55  
56  
57  
58  
59  
60  
Run | Debug  
public static void main(String[] args) {  
    int arr[] = {5, 2, 4, 7, 1};  
    MergeSort(arr, left:0, arr.length - 1);  
  
    System.out.print(s:"Sorted Array: ");  
    for (int n : arr) {  
        System.out.print(n + " ");  
    }  
}
```

Output:

```
[Running] cd "d:\DSA\code\" && javac CountingSort.java && java CountingSort  
Sorted Array: 2 2 3 4 8  
[Done] exited with code=0 in 2.99 seconds
```

◆ 4. Selection Sort (Review Only)

✦ **Explanation:** Selection Sort works by repeatedly finding the smallest element from the unsorted part of the array and placing it at the correct position in the sorted part.

Example:

Input: [5, 3, 4, 1]

Step 1: Smallest element is 1 → [1, 3, 4, 5]

Step 2: Array becomes sorted → [1, 3, 4, 5]

◆ Time Complexity:

- Best Case: $O(n^2)$
- Worst Case: $O(n^2)$
- Average Case: $O(n^2)$
- Space Complexity: $O(1)$

◆ 5. Quick Sort (Review Only)

◆ **Explanation:** Quick Sort is another Divide and Conquer algorithm.

- A pivot is chosen.
- Elements smaller than pivot are placed on the left, larger on the right.
- The process is applied recursively to the left and right partitions.

Example:

Input: [5, 3, 7, 2]

Pivot = 5 $\rightarrow$ Left = [3, 2], Right = [7] $\rightarrow$ Final = [2, 3, 5, 7]

◆ Time Complexity:

- Best Case: $O(n \log n)$
- Worst Case: $O(n^2)$ (if pivot is always bad)
- Average Case: $O(n \log n)$
- Space Complexity: $O(\log n)$